🏃 Personal Running Coach Agent Swarm

Course Demonstration Notebook

Student: Will Sutherland | **Course:** Agentic AI | **Date:** May 2026

GitHub: https://github.com/willsuther/running-coach-agent-swarm

---

This notebook demonstrates the complete Personal Running Coach Agent Swarm — a multi-agent AI system built to support half-marathon training. All outputs shown are generated from real personal training data.

What this notebook demonstrates

Section	Agent	Capability demonstrated
1	Setup	Environment, tools, data pipeline
2	Feedback Agent	Workout analysis, pace evaluation, RAG, lap splits
3	Recovery Agent	HRV, sleep, body battery, TSB, session recommendation
4	Planner Agent	Taper-aware week generation, recovery-informed planning
5	Coordinator Agent	Natural language routing, multi-agent synthesis, self-critique
6	Safeguards	Injury disclaimer, [RECOVERY ALERT], mileage rule
7	Memory / RAG	ChromaDB retrieval across three collections

Section 1 — Environment Setup

Install dependencies, mount Google Drive, load API credentials, and import the tool layer. All data lives in Google Drive — no data is hardcoded or bundled with the notebook.

```
%pip install -q chromadb google-generativeai langchain langchain-google-genai langgraph stravalib gradio

import os, sys, sqlite3
from google.colab import drive, userdata

drive.mount('/content/drive', force_remount=False)

BASE_DIR       = "/content/drive/MyDrive/running_coach"
GEMINI_API_KEY = userdata.get('key')

sys.path.insert(0, f"{BASE_DIR}/tools")
sys.path.insert(0, f"{BASE_DIR}/agents")

# ── Verify all components present ──────────────────────────────
components = {
    'garmin.db':               f"{BASE_DIR}/data/raw/garmin/garmin.db",
    'workouts_normalized.json':f"{BASE_DIR}/data/processed/workouts_normalized.json",
    'ChromaDB memory':         f"{BASE_DIR}/memory/chroma",
    'parse_workout_data':      f"{BASE_DIR}/tools/parse_workout_data.py",
    'calculate_pace_zones':    f"{BASE_DIR}/tools/calculate_pace_zones.py",
    'calculate_training_load': f"{BASE_DIR}/tools/calculate_training_load.py",
    'query_garmin_db':         f"{BASE_DIR}/tools/query_garmin_db.py",
    'memory_retrieval':        f"{BASE_DIR}/tools/memory_retrieval.py",
    'feedback_agent':          f"{BASE_DIR}/agents/feedback_agent.py",
    'recovery_agent':          f"{BASE_DIR}/agents/recovery_agent.py",
    'planner_agent':           f"{BASE_DIR}/agents/planner_agent.py",
    'coordinator_agent':       f"{BASE_DIR}/agents/coordinator_agent.py",
}

all_ok = True
for name, path in components.items():
    exists = os.path.exists(path)
    print(f"  {'✅' if exists else '❌'} {name}")
    if not exists:
        all_ok = False

print(f"\n{'✅ All components ready.' if all_ok else '❌ Missing components — check Drive.'}")
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 52.0/52.0 kB 3.1 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 23.3/23.3 MB 67.7 MB/s eta 0:00:00
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 67.6/67.6 kB 5.7 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 125.4/125.4 kB 10.5 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 278.2/278.2 kB 17.0 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4.7/4.7 MB 87.3 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 18.2/18.2 MB 78.6 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 71.8/71.8 kB 5.2 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 170.9/170.9 kB 14.3 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 61.3/61.3 kB 4.6 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 203.7/203.7 kB 15.0 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 71.6/71.6 kB 5.9 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 60.6/60.6 kB 4.7 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 307.5/307.5 kB 24.8 MB/s eta 0:00:00
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the
opentelemetry-exporter-gcp-logging 1.11.0a0 requires opentelemetry-sdk<1.39.0,>=1.35.0, but you have opentelemetry-sdk 1.42.0 wh
google-adk 1.29.0 requires opentelemetry-api<1.39.0,>=1.36.0, but you have opentelemetry-api 1.42.0 which is incompatible.
google-adk 1.29.0 requires opentelemetry-sdk<1.39.0,>=1.36.0, but you have opentelemetry-sdk 1.42.0 which is incompatible.
opentelemetry-exporter-otlp-proto-http 1.38.0 requires opentelemetry-exporter-otlp-proto-common==1.38.0, but you have opentelemet
opentelemetry-exporter-otlp-proto-http 1.38.0 requires opentelemetry-proto==1.38.0, but you have opentelemetry-proto 1.42.0 whic
opentelemetry-exporter-otlp-proto-http 1.38.0 requires opentelemetry-sdk~=1.38.0, but you have opentelemetry-sdk 1.42.0 which is
Mounted at /content/drive
  ✅ garmin.db
  ✅ workouts_normalized.json
  ✅ ChromaDB memory
  ✅ parse_workout_data
  ✅ calculate_pace_zones
  ✅ calculate_training_load
  ✅ query_garmin_db
  ✅ memory_retrieval
  ✅ feedback_agent
  ✅ recovery_agent
  ✅ planner_agent
  ✅ coordinator_agent

✅ All components ready.
```

Data pipeline summary

The system combines two data sources:

- **`garmin.db`** — SQLite database generated locally via `garmin-givemydata` (headless Chrome to bypass Garmin's Cloudflare TLS protection). Contains 807 activities, 843 HRV records, 819 sleep records, 3,651 days of body battery and stress data.
- **Strava API** — OAuth pull via `stravalib`. Provides activity list and suffer scores.

Both sources are normalized to a shared `WorkoutRecord` schema during Phase 1, enriched with Garmin physiological fields matched by date and start time.

```
import pandas as pd
from parse_workout_data import load_workouts, summarise_workouts, format_pace
from calculate_pace_zones import classify_workouts

workouts = load_workouts(f"{BASE_DIR}/data/processed/workouts_normalized.json", days=180)
enriched = classify_workouts(workouts)
summary  = summarise_workouts(workouts)

print("=== Workout Dataset (last 180 days) ===")
print(f"  Total runs:              {summary['count']}")
print(f"  Total distance:          {summary['total_distance_km']} km")
print(f"  Average pace:            {summary['avg_pace_formatted']}")
print(f"  Average HR:              {summary['avg_hr']} bpm")
print(f"  Garmin-enriched records: {summary['garmin_enriched_count']} / {summary['count']}")
print(f"  Date range:              {summary['date_range_start']} → {summary['date_range_end']}")

structured  = [w for w in enriched if w['classification']['is_structured']]
easy        = [w for w in enriched if w['classification']['workout_type'] == 'easy']
needs_input = [w for w in enriched if w['classification']['needs_input']]

print(f"\n=== Workout Classification ===")
print(f"  Structured (Garmin date-named): {len(structured)}")
print(f"  Easy (auto-classified):         {len(easy)}")
print(f"  Needs user input:               {len(needs_input)}")

# Garmin DB summary
conn = sqlite3.connect(f"{BASE_DIR}/data/raw/garmin/garmin.db")
print(f"\n=== Garmin DB ===")
for table in ['activity', 'hrv', 'sleep', 'body_battery', 'training_readiness', 'race_predictions']:
    count = conn.execute(f"SELECT COUNT(*) FROM {table}").fetchone()[0]
```

```
        print(f"  {table:<22} {count:>6,} rows")
conn.close()
```

```
=== Workout Dataset (last 180 days) ===
  Total runs:              127
  Total distance:          1300.54 km
  Average pace:            4:54/km
  Average HR:              138.82 bpm
  Garmin-enriched records: 124 / 127
  Date range:              2025-11-22 → 2026-05-20

=== Workout Classification ===
  Structured (Garmin date-named): 36
  Easy (auto-classified):         88
  Needs user input:               3

=== Garmin DB ===
  activity                  829 rows
  hrv                       864 rows
  sleep                     838 rows
  body_battery            3,673 rows
  training_readiness        872 rows
  race_predictions          867 rows
```

Section 2 — Feedback Agent

Demonstrates: Tool use (`parse_workout_data`, `calculate_pace_zones`, `query_garmin_db`), RAG memory retrieval, lap split analysis, pace evaluation against coach-prescribed targets, injury disclaimer safeguard.

The Feedback Agent analyses a completed workout by:

1. Loading and classifying the WorkoutRecord
2. Evaluating actual pace against the coach-prescribed target (±5 second tolerance)
3. Pulling lap splits from `garmin.db` to see interval-level performance
4. Retrieving similar past sessions from ChromaDB via semantic search
5. Generating structured coaching feedback with Gemini 2.5 Flash

```
# ── Tool layer demo: pace zone classification ──────────────────────────
print('=== Tool: calculate_pace_zones — Workout Classification ===')
print('\nCoach-prescribed targets (±5 second tolerance):')
from calculate_pace_zones import TRAINING_PACES, format_pace as fmt, PACE_TOLERANCE_SEC
from query_garmin_db import get_activity_splits
for k, v in TRAINING_PACES.items():
    if v:
        print(f'  {k:<12} {fmt(v)}')
    else:
        print(f'  {k:<12} by feel (ceiling 4:45/km)')

print(f"\n{'Date':<12} {'Type':<12} {'Effort Pace':<12} {'Target':<10} {'Diff':>6} {'Status'}")
print('-' * 68)

def get_hard_effort_pace(date_str, conn):
    """Return avg pace of hard effort laps, or None if no splits."""
    try:
        row = conn.execute("""
            SELECT activity_id FROM activity
            WHERE DATE(start_time_local) = ?
              AND LOWER(activity_type) LIKE '%run%'
            ORDER BY start_time_local DESC LIMIT 1
        """, (date_str,)).fetchone()
        if not row: return None
        splits = get_activity_splits(conn, row[0])
        if not splits: return None
        full_km = [s for s in splits if s['distance_m'] >= 900]
        if not full_km: return None
        avg_s_per_km = sum(s['duration_s']/(s['distance_m']/1000) for s in full_km) / len(full_km)
        hard = [s for s in splits if s['distance_m'] >= 200 and
                s['duration_s']/(s['distance_m']/1000) < avg_s_per_km - 15]
        if not hard: return None
        avg_hard = sum(s['duration_s']/(s['distance_m']/1000)/60 for s in hard) / len(hard)
        return avg_hard
    except:
        return None
```

```
conn = sqlite3.connect(f"{BASE_DIR}/data/raw/garmin/garmin.db")
resolved = [w for w in enriched if w['classification']['workout_type'] and not w['classification']['needs_input']][:4]
pending  = [w for w in enriched if w['classification']['detected_from'] == 'garmin_name'][:2]
# Add April 8 VO2 max exception if present
exception = next((w for w in enriched if w.get('date') == '2026-04-08'), None)
sample = ([exception] if exception else []) + [w for w in resolved if w.get('date') != '2026-04-08'][:3] + pending[:2]

for w in sample:
    pe     = w.get('pace_evaluation', {})
    wtype  = w['classification'].get('workout_type') or 'pending'
    target = pe.get('target_pace_fmt', 'N/A') if pe else 'N/A'

    # For structured sessions use hard effort pace from splits
    if wtype not in ('easy', 'unclassified', 'pending', None):
        hard_pace = get_hard_effort_pace(w['date'], conn)
        if hard_pace:
            actual   = fmt(hard_pace)
            target_v = TRAINING_PACES.get(wtype)
            if target_v:
                diff_s = (hard_pace - target_v) * 60
                diff   = f"{diff_s:+.0f}s"
                on_t   = abs(diff_s) <= PACE_TOLERANCE_SEC
                status = '✅ on target (rep pace)' if on_t else ('⚠️  off target (rep pace)' if diff_s > 0 else '✅ faster th
            else:
                diff, status = 'N/A', '—'
        else:
            actual = pe.get('actual_pace_fmt', '--:--') if pe else '--:--'
            diff, status = 'N/A', '— (no splits)'
    elif wtype == 'easy':
        actual = pe.get('actual_pace_fmt', '--:--') if pe else '--:--'
        diff, status = 'N/A', '— easy (no target)'
    else:
        actual = pe.get('actual_pace_fmt', '--:--') if pe else '--:--'
        diff, status = 'N/A', '⏳ type pending (Planner resolves)'

    print(f"{w['date']:<12} {wtype:<12} {actual:<12} {target:<10} {diff:>6} {status}")

conn.close()
print('\nNote: structured sessions show hard effort (rep) pace, not overall average.')
print('Easy runs use overall pace vs 4:45/km ceiling. Pending = awaiting Planner resolution.')
```

```
=== Tool: calculate_pace_zones — Workout Classification ===

Coach-prescribed targets (±5 second tolerance):
  easy         by feel (ceiling 4:45/km)
  marathon     4:14/km
  threshold    4:01/km
  1hr          3:56/km
  fartlek      3:49/km
  8k           3:46/km
  vo2max       3:40/km

Date         Type         Effort Pace  Target       Diff Status
----------------------------------------------------------------------
2026-04-08   vo2max       3:31/km      3:40/km       -9s ✅ faster than target (rep pace)
2026-05-20   easy         5:08/km      by feel       N/A — easy (no target)
2026-05-17   easy         4:32/km      by feel       N/A — easy (no target)
2026-05-17   easy         3:43/km      by feel       N/A — easy (no target)
2026-05-06   pending      4:16/km      --:--/km      N/A ⏳ type pending (Planner resolves)
2026-05-02   pending      4:10/km      --:--/km      N/A ⏳ type pending (Planner resolves)

Note: structured sessions show hard effort (rep) pace, not overall average.
Easy runs use overall pace vs 4:45/km ceiling. Pending = awaiting Planner resolution.
```

```
# ── Tool layer demo: lap splits from Garmin DB ──────────────────────────
from query_garmin_db import get_activity_splits, format_splits

print("=== Tool: query_garmin_db — Lap Splits ===")

conn = sqlite3.connect(f"{BASE_DIR}/data/raw/garmin/garmin.db")
row  = conn.execute("""
    SELECT activity_id, activity_name, DATE(start_time_local)
    FROM activity
    WHERE LOWER(activity_type) LIKE '%run%' and DATE(start_time_local) = '2026-05-09'
    ORDER BY start_time_local DESC LIMIT 1
""").fetchone()

print(f"\nActivity: {row[1]} | Date: {row[2]}")
```

```
splits = get_activity_splits(conn, row[0])
print(f"{len(splits)} splits found\n")
print(format_splits(splits))
conn.close()
```

```
=== Tool: query_garmin_db — Lap Splits ===

Activity: Halifax - Shakeout | Date: 2026-05-09
12 splits found

Split  Distance    Duration  Pace       Avg HR  Max HR  Elev   Note
--------------------------------------------------------------------------
  1     1.00km      4:30      4:31/km    128.0   143.0   20.0
  2     1.00km      4:29      4:30/km    135.0   145.0   13.0
  3     1.00km      4:32      4:33/km    143.0   149.0   10.0
  4     1.00km      4:57      4:58/km    141.0   151.0   4.0    🔄 easy
  5     319m        1:29      4:40/km    142.0   149.0   5.0
  6     100m        0:19      3:15/km    144.0   151.0   6.0    ⚡ hard
  7     202m        1:03      5:13/km    152.0   156.0   1.0    🔄 easy
  8     100m        0:18      3:09/km    145.0   149.0   0      ⚡ hard
  9     197m        0:58      4:58/km    151.0   155.0   3.0    🔄 easy
  10    100m        0:18      3:08/km    150.0   155.0   0      ⚡ hard
  11    104m        0:30      4:55/km    158.0   160.0   0      🔄 easy
  12    100m        0:17      2:55/km    155.0   159.0   2.0    ⚡ hard
```

```
# ── Feedback Agent: full run ──────────────────────────────────────────
print("=== Feedback Agent — Most Recent Run ===")
print("Generating feedback...\n")

import google.generativeai as genai
from calculate_training_load import get_metrics_for_date
from calculate_training_load import get_metrics_for_date

genai.configure(api_key=GEMINI_API_KEY)
model = genai.GenerativeModel("gemini-2.5-flash")

# Custom embedder — avoids google-api-core ClientOptions conflict
import chromadb, requests
class GeminiEmbedder:
    def __init__(self, api_key, model='models/gemini-embedding-001'):
        self.api_key = api_key
        self.model   = model
        self.name    = 'gemini-embedder'
    def __call__(self, input):
        embeddings = []
        for text in input:
            url  = f'https://generativelanguage.googleapis.com/v1beta/{self.model}:embedContent?key={self.api_key}'
            resp = requests.post(url, json={'model': self.model, 'content': {'parts': [{'text': text}]}})
            embeddings.append(resp.json()['embedding']['values'])
        return embeddings

mem_client = chromadb.PersistentClient(path=f"{BASE_DIR}/memory/chroma")
mem_ef     = GeminiEmbedder(GEMINI_API_KEY)

workout = enriched[0]
cl      = workout.get('classification', {})
pe      = workout.get('pace_evaluation', {})
hc      = workout.get('health_context', {})
wtype   = cl.get('workout_type') or 'unclassified'

ctx = [
    f"Date: {workout['date']} | Type: {wtype}",
    f"Distance: {workout.get('distance_km')}km | Duration: {workout.get('duration_min')}min",
    f"Avg pace: {format_pace(workout.get('avg_pace_min_km'))} | Avg HR: {workout.get('avg_hr')} bpm | Max HR: {workout.get('max
    f"Training load: {workout.get('training_load')} | Aerobic TE: {workout.get('aerobic_effect')} | Suffer score: {workout.get(
]

if pe and pe.get('verdict'):
    ctx.append(f"Pace verdict: {pe['verdict']}")

if hc:
    parts = []
    if hc.get('hrv_status'):            parts.append(f"HRV: {hc['hrv_status']}")
    if hc.get('sleep_sleep_time_seconds'): parts.append(f"Sleep: {round(hc['sleep_sleep_time_seconds']/3600,1)}h")
    if hc.get('training_readiness_score'): parts.append(f"Readiness: {hc['training_readiness_score']}")
    if parts:
        ctx.append(f"Recovery on day: {', '.join(parts)}")
```

```
try:
    m = get_metrics_for_date(workouts, workout['date'])
    if m:
        ctx.append(f"TSB on day: {m.tsb:.1f} ({m.form_label})")
except Exception:
    pass

# Add splits
conn = sqlite3.connect(f"{BASE_DIR}/data/raw/garmin/garmin.db")
row  = conn.execute("""
    SELECT activity_id FROM activity
    WHERE DATE(start_time_local) = ?
      AND LOWER(activity_type) LIKE '%run%'
    ORDER BY start_time_local DESC LIMIT 1
""", (workout['date'],)).fetchone()
if row:
    splits = get_activity_splits(conn, row[0])
    if splits:
        ctx += ["Lap splits:", format_splits(splits)]
conn.close()

# RAG: similar past sessions
try:
    # Embed the query manually then search
    url      = f'https://generativelanguage.googleapis.com/v1beta/models/gemini-embedding-001:embedContent?key={GEMINI_API_KEY}
    resp     = requests.post(url, json={'model': 'models/gemini-embedding-001', 'content': {'parts': [{'text': f'{wtype} sessic
    qvec     = resp.json()['embedding']['values']
    col      = mem_client.get_collection('workout_summaries')
    results  = col.query(query_embeddings=[qvec], n_results=2)
    docs     = results['documents'][0] if results['documents'] else []
    metas    = results['metadatas'][0] if results['metadatas'] else []
    if docs:
        ctx.append('Similar past sessions:')
        for doc, meta in zip(docs, metas):
            if meta.get('date') != workout['date']:
                ctx.append(doc.split('\n')[0])
except Exception as e:
    print(f'  (RAG skipped: {e})')

system = """You are a running coach for Will Sutherland (sub-1:24 HM target).
Format:
**Numbers** - [pace, target, HR, load, TSB]
**How it went** - [2-3 sentences]
**Patterns** - [1-2 sentences vs similar sessions]
**Next session consideration** - [1 sentence]
Under 200 words. Direct. Real numbers."""

response = model.generate_content(f"{system}\n\nWorkout data:\n" + "\n".join(ctx))
print(response.text)
```

```
=== Feedback Agent — Most Recent Run ===
Generating feedback...

/usr/local/lib/python3.12/dist-packages/google/colab/_import_hooks/_hook_injector.py:55: FutureWarning:

All support for the `google.generativeai` package has ended. It will no longer be receiving
updates or bug fixes. Please switch to the `google.genai` package as soon as possible.
See README for more details:

https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md

  loader.exec_module(module)
  ℹ️  3 record(s) skipped (no training_load — not Garmin-enriched). Treated as 0 load.
**Numbers** - [5:08/km, >4:45/km (Easy), 121.3 bpm, Minimal Load (Suffer 8), 48.1]
**How it went** - Will, this 8.03km easy run was perfectly executed. Your 5:08/km pace and average HR of 121.3 bpm confirm it wa
**Patterns** - This run shows excellent consistency with previous easy efforts, maintaining paces around 5:06-5:12/km. It highli
**Next session consideration** - With peak freshness, you are well-prepared for your next planned quality workout.
```

Section 3 — Recovery Agent

Demonstrates: Multi-signal recovery assessment (HRV, sleep, body battery, TSB), PROCEED/MODIFY/BACK OFF recommendation, [RECOVERY ALERT] safeguard, training readiness from `garmin.db`.

The Recovery Agent pulls three days of Garmin physiological data alongside ATL/CTL/TSB metrics to produce a daily recovery assessment and pre-workout readiness recommendation.

```
# ── Tool layer demo: Garmin health signals ──────────────────────────────
from query_garmin_db import get_recent_snapshots, format_recovery_context

print("=== Tool: query_garmin_db — Recovery Signals (last 21 days) ===")
conn      = sqlite3.connect(f"{BASE_DIR}/data/raw/garmin/garmin.db")
snapshots = get_recent_snapshots(conn, days=21)
conn.close()
print(format_recovery_context(snapshots, include_days=21))
```

```
    Sleep history: MODERATE
  HRV: None (weekly avg: 100.0) — Balanced ✅, baseline N/A
  Sleep: 8.08h total (deep: 1.98h, REM: 1.02h) — Good duration
    Feedback: POSITIVE_LONG_AND_DEEP
  Body battery at wake: 43 — Partially recovered ⚠️ (peak: 96, low: 43)
  Stress: avg 18, max 92 — Unknown
  Resting HR: 43 bpm

--- 2026-05-03 ---
  Recovery score (composite): 72/100
  Training readiness: 68.0 — Moderate
    Feedback: GOOD_SLEEP_HISTORY
    HRV factor: VERY_GOOD
    Sleep history: GOOD
  HRV: None (weekly avg: 100.0) — Balanced ✅, baseline N/A
  Body battery at wake: 69 — Adequately recovered (peak: 69, low: 34)
  Stress: avg 22, max 91 — Unknown
  Resting HR: 42 bpm

--- 2026-05-02 ---
  Recovery score (composite): 64/100
  Training readiness: 77.0 — Moderate
    Feedback: WELL_RECOVERED
    HRV factor: GOOD
    Sleep history: GOOD
  HRV: None (weekly avg: 97.0) — Balanced ✅, baseline N/A
  Sleep: 7.18h total (deep: 1.0h, REM: 0.67h) — Adequate duration
    Feedback: NEGATIVE_LONG_BUT_NOT_ENOUGH_REM
  Body battery at wake: 32 — Partially recovered ⚠️ (peak: 94, low: 22)
  Stress: avg 29, max 98 — Unknown
  Resting HR: 41 bpm

--- 2026-05-01 ---
  Recovery score (composite): 61/100
  Training readiness: 73.0 — Moderate
    Feedback: RECOVERED_AND_READY
    Recovery time remaining: 1h
    HRV factor: GOOD
    Sleep history: GOOD
  HRV: None (weekly avg: 92.0) — Balanced ✅, baseline N/A
  Sleep: 7.6h total (deep: 1.63h, REM: 1.22h) — Good duration
    Feedback: POSITIVE_LONG_AND_DEEP
  Body battery at wake: 26 — Poorly recovered ⚠️ (peak: 87, low: 26)
  Stress: avg 24, max 95 — Unknown
  Resting HR: 39 bpm

--- 2026-04-30 ---
  Recovery score (composite): 64/100
  Training readiness: 68.0 — Moderate
    Feedback: RECOVERED_AND_READY
    HRV factor: GOOD
    Sleep history: GOOD
  HRV: None (weekly avg: 94.0) — Balanced ✅, baseline N/A
  Sleep: 6.81h total (deep: 1.88h, REM: 1.57h) — Adequate duration
    Feedback: POSITIVE_DEEP
  Body battery at wake: 41 — Partially recovered ⚠️ (peak: 89, low: 22)
  Stress: avg 21, max 91 — Unknown
  Resting HR: 40 bpm
```

```
# ── Tool layer demo: Training load ──────────────────────────────────────
from calculate_training_load import (
    get_current_metrics, get_recent_trend,
    check_recovery_alert, weekly_load_summary,
    check_mileage_rule, format_metrics_report
)

print("=== Tool: calculate_training_load ===")
metrics = get_current_metrics(workouts)
trend   = get_recent_trend(workouts, days=21)
```

```
alert   = check_recovery_alert(workouts)
mileage = check_mileage_rule(workouts)
print(format_metrics_report(metrics, trend, alert, mileage))
```

```
=== Tool: calculate_training_load ===
=== Training Load Report — 2026-05-20 ===
  ATL (fatigue):   27.1
  CTL (fitness):   75.2
  TSB (form):      48.1  →  Very fresh — peak form
  Fitness level:   Good fitness

  14-day trend:
    ATL:  88.9 → 27.1  (falling)
    CTL:  101.8 → 75.2  (falling)
    TSB:  12.9 → 48.1  (rising)
    Total load:     730
    Avg daily load: 34.8
    Peak load:      214.6
    Rest days:      11 / 21

  Mileage rule: ℹ️  Week just started (day 3/7) — no sessions logged yet. Last week total load: 166. 10% rule target for this we
```

```
# ── Recovery Agent: daily check-in ──────────────────────────────────────
print("=== Recovery Agent — Daily Check-In ===")
print("Generating recovery assessment...\n")

from recovery_agent import SYSTEM_PROMPT_CHECKIN

# Build context
ctx_lines = []
conn      = sqlite3.connect(f"{BASE_DIR}/data/raw/garmin/garmin.db")
snapshots = get_recent_snapshots(conn, days=3)
conn.close()
ctx_lines.append(format_recovery_context(snapshots, include_days=21))
ctx_lines.append(format_metrics_report(metrics, trend, alert, mileage))

try:
    col     = mem_client.get_collection('athlete_profile', embedding_function=mem_ef)
    res     = col.query(query_texts=['recovery readiness training load'], n_results=2)
    profile = '\n\n'.join(res['documents'][0]) if res['documents'] else ''
except Exception:
    profile = ''
if profile:
    ctx_lines += ["\nAthlete context:", profile]

context  = "\n".join(ctx_lines)
response = model.generate_content(f"{SYSTEM_PROMPT_CHECKIN}\n\nData:\n{context}")
print(response.text)
```

```
=== Recovery Agent — Daily Check-In ===
Generating recovery assessment...

**Recovery Status: MODERATE**

**Key signals**
- [HRV: None, status: Low ⚠️, vs baseline: N/A (weekly avg: 82.0)]
- [Sleep: 7.65h, quality: Good duration]
- [Body battery at wake: 6, label: Poorly recovered ⚠️]
- [Training readiness: 1.0, level: Very low ⚠️]
- [TSB (form): 48.1, label: Very fresh — peak form]
- [Resting HR: 46 bpm]

**Assessment**
Your daily physiological recovery metrics from May 18th (HRV status, Body Battery 6, Training Readiness 1.0) all indicated a sig

**Recommendation**
Given your TSB indicates peak form, you are likely prepared for today's planned session. However, remain highly attuned to your
```

Section 4 — Planner Agent

Demonstrates: Race-aware taper logic, weekly schedule generation, 10% mileage rule enforcement, recovery-informed session modification.

The Planner Agent automatically detects the current training phase based on days to race and adjusts recommendations accordingly. With the Fredericton Half Marathon on May 10th (using a dummy date of June 10th for demo), it enters race week taper logic.

```
from datetime import date
import time

RACE_DATE = date(2026, 6, 10)
days_left = (RACE_DATE - date.today()).days
phase = "Taper — week 2" if days_left <= 14 else "Build"

print(f"=== Planner Agent — Week Plan ===")
print(f"Toronto Half Marathon | {days_left} days to go | Phase: {phase}\n")

ctx = f"""Race: Toronto Half Marathon, {days_left} days away. Phase: {phase}.
ATL: {metrics.atl:.1f}, CTL: {metrics.ctl:.1f}, TSB: {metrics.tsb:.1f} ({metrics.form_label})
Training days: Tue/Wed/Thu/Sat/Sun. Rest: Mon/Fri.
Wednesday = key speed session. Saturday = long run with structure.
Paces: easy <4:45/km, marathon 4:14, threshold 4:01, HM target ~3:59/km."""

prompt = """You are a running coach for Will Sutherland (sub-1:24 HM target).
Generate a weekly training plan as a markdown table with columns: Day | Session | Details | Notes.
Include weekly volume and a 2-sentence rationale. Under 150 words total."""

for attempt in range(3):
    try:
        response = model.generate_content(f"{prompt}\n\nContext:\n{ctx}")
        print(response.text)
        break
    except Exception as e:
        print(f"Attempt {attempt+1} failed: {e}")
        time.sleep(5)
```

```
=== Planner Agent — Week Plan ===
Toronto Half Marathon | 21 days to go | Phase: Build

Here's Will's training plan for the build phase, 21 days out from race day:

| Day | Session | Details | Notes |
| :-- | :------ | :------ | :---- |
| Mon | Rest | Full recovery day. | Prioritize sleep and active recovery (e.g., light stretching). |
| Tue | Easy Run + Strides | 8km easy (<4:45/km). Finish with 6x100m strides, jog recovery. | Focus on relaxed form and maintain
| Wed | HM Pace Intervals | Warm-up: 2km easy. Workout: 4 x 2km at HM pace (3:59/km) with 3min easy recovery. Cool-down: 2km eas
| Thu | Medium Easy Run | 12km easy (<4:45/km). | Focus on active recovery and aerobic development. |
| Fri | Rest | Full recovery day. | Prepare mentally and physically for your long run. |
| Sat | Structured Long Run | 20km total: 6km easy, then 8km at Marathon pace (4:14/km), then 4km at HM pace (3:59/km), then 2km
| Sun | Easy Long Run | 14km easy (<4:45/km). | Focus on relaxed running, enjoy the miles, and allow for recovery. |

**Weekly Volume:** 68km

This week maximizes race-specific fitness and confidence without accumulating excessive fatigue, given the extremely high TSB an
```

Section 5 — Coordinator Agent

Demonstrates: Natural language query routing, multi-agent orchestration, response synthesis, self-critique safety pass.

The Coordinator is the user-facing entry point. It classifies queries via keyword matching, dispatches to the appropriate specialist agent(s), synthesizes multi-agent outputs, and runs a safety review before delivery.

Example 1: Single-agent routing (Feedback)

```
from coordinator_agent import CoordinatorAgent

coordinator = CoordinatorAgent(api_key=GEMINI_API_KEY)

print("Query: 'How did my most recent run go?'")
print("Expected routing: → Feedback Agent\n")
coordinator.run(query="How did my most recent run go?")
```

```
✅ Coordinator Agent initialized.
   Specialist agents: Feedback | Recovery | Planner
Query: 'How did my most recent run go?'
Expected routing: → Feedback Agent


📨 Query: How did my most recent run go?

🧠 Classifying query...

   Routing to: ['feedback']

============================================================
RUNNING COACH
============================================================
   📊 Calling Feedback Agent...
   🔀 Synthesizing responses...
   🔍 Running self-critique pass...
   Self-critique: ✅ APPROVED

**Numbers** - Pace 5:08/km (Target easy: >4:45/km), Avg HR 121.3 bpm, Load Low (Suffer 8), TSB 48.1 (Very Fresh).

**How it went** - Will, this was an excellent easy run, perfectly executed at 5:08/km, well within your easy zone. Your average

**Patterns** - This pace and HR are consistent with your most disciplined recovery runs, showing great control. You are effectiv

**Next session consideration** - Continue this disciplined approach to easy runs to ensure full recovery and build your aerobic

------------------------------------------------------------
Ask a follow-up, choose another option, or type 'done'.
------------------------------------------------------------

You: done

Closing Running Coach. Good luck with your training! 🏃
```

Example 2: Multi-agent routing (Feedback + Recovery)

This query spans two domains — the Coordinator detects it needs both the Feedback Agent (how did the run go?) and the Recovery Agent (should I do tomorrow's session?), calls both, and synthesizes a single coherent response.

```
print("Query: 'How did my run yesterday go and should I do tomorrow's threshold session?'")
print("Expected routing: → Feedback Agent + Recovery Agent\n")

coordinator.run(query="How did my run yesterday go and should I do tomorrow's threshold session?")
```

```
Query: 'How did my run yesterday go and should I do tomorrow's threshold session?'
Expected routing: → Feedback Agent + Recovery Agent


📨 Query: How did my run yesterday go and should I do tomorrow's threshold session?

🧠 Classifying query...

   Routing to: ['feedback', 'recovery']

============================================================
RUNNING COACH
============================================================
   📊 Calling Feedback Agent...
   💤 Calling Recovery Agent...
   🔀 Synthesizing responses...
   🔍 Running self-critique pass...
   Self-critique: ✅ APPROVED

Will, your recent 8km easy run was executed exceptionally well at a 5:08/km pace, maintaining a very controlled average heart ra

Crucially, your current TSB stands at 48.1. This indicates you are exceptionally fresh, fully recovered, and in peak form. Both

Therefore, **PROCEED** as planned with your threshold session at 4:01/km. You are in an optimal state to execute this workout ef

------------------------------------------------------------
Ask a follow-up, choose another option, or type 'done'.
------------------------------------------------------------

You: done

Closing Running Coach. Good luck with your training! 🏃
```

Section 6 — Safeguards

Demonstrates: Three safeguards from the project specification.

Safeguard 1: Injury disclaimer intercept

Any mention of pain or injury keywords triggers an immediate disclaimer redirecting to a human professional, regardless of context.

```
from coordinator_agent import INJURY_KEYWORDS, INJURY_DISCLAIMER

test_queries = [
    "My knee has been a bit sore after long runs, should I still do Saturday?",
    "I have some hamstring tightness, is it safe to do tomorrow's threshold?",
    "How did my run go today?",   # Should NOT trigger
]

print("=== Safeguard 1: Injury Disclaimer Intercept ===")
for q in test_queries:
    triggered = any(kw in q.lower() for kw in INJURY_KEYWORDS)
    print(f"\nQuery: '{q}'")
    if triggered:
        print(f"→ INTERCEPTED: {INJURY_DISCLAIMER}")
    else:
        print("→ No injury keywords detected — query proceeds normally.")
```

```
=== Safeguard 1: Injury Disclaimer Intercept ===

Query: 'My knee has been a bit sore after long runs, should I still do Saturday?'
→ INTERCEPTED:
⚠️  INJURY DISCLAIMER: Your query mentions a possible physical symptom. Please consult your coach or a physiotherapist before c

Query: 'I have some hamstring tightness, is it safe to do tomorrow's threshold?'
→ INTERCEPTED:
⚠️  INJURY DISCLAIMER: Your query mentions a possible physical symptom. Please consult your coach or a physiotherapist before c

Query: 'How did my run go today?'
→ No injury keywords detected — query proceeds normally.
```

Safeguard 2: [RECOVERY ALERT] when TSB < -30

```
from calculate_training_load import TSB_ALERT_THRESHOLD

print("=== Safeguard 2: [RECOVERY ALERT] TSB Threshold ===")
print(f"Alert threshold: TSB < {TSB_ALERT_THRESHOLD}")
print(f"Current TSB: {metrics.tsb:.1f} ({metrics.form_label})")

alert = check_recovery_alert(workouts)
if alert:
    print(f"\n{alert.message}")
else:
    print(f"\n✅ No alert — TSB is within acceptable range.")
    print("\nSimulated alert (what fires when TSB < -30):")
    print("[RECOVERY ALERT] TSB is -32.4 (threshold: -30). TSB has been below "
          "threshold for 3 consecutive day(s). Current fatigue (ATL): 118.2. "
          "Fitness (CTL): 85.8. Recommend reduced load or rest day before next hard session.")
```

```
=== Safeguard 2: [RECOVERY ALERT] TSB Threshold ===
Alert threshold: TSB < -30
Current TSB: 48.1 (Very fresh — peak form)

✅ No alert — TSB is within acceptable range.

Simulated alert (what fires when TSB < -30):
[RECOVERY ALERT] TSB is -32.4 (threshold: -30). TSB has been below threshold for 3 consecutive day(s). Current fatigue (ATL): 11
```

Safeguard 3: 10% mileage rule with week projection

```
print("=== Safeguard 3: 10% Mileage Rule ===")
print(f"Rule: weekly load increase capped at {10}%")
print(f"Incomplete week logic: projects full-week load from days elapsed\n")
print(mileage['message'])

weeks = weekly_load_summary(workouts, weeks=6)
print(f"\n6-week load history:")
print(f"  {'Week':<12} {'Load':>8} {'Sessions':>9} {'Avg TSB':>8} {'Change':>8}")
print("  " + "-" * 50)
for w in weeks:
    change = f"{w.pct_change_load:+.1f}%" if w.pct_change_load is not None else "  base"
    flag   = " ⚠️" if w.pct_change_load and w.pct_change_load > 10 else ""
    print(f"  {w.week_start:<12} {w.total_load:>8.0f} {w.session_count:>9} {w.avg_tsb:>8.1f} {change:>8}{flag}")
```

```
=== Safeguard 3: 10% Mileage Rule ===
Rule: weekly load increase capped at 10%
Incomplete week logic: projects full-week load from days elapsed

ℹ️  Week just started (day 3/7) — no sessions logged yet. Last week total load: 166. 10% rule target for this week: ≤ 183.

6-week load history:
  Week             Load  Sessions  Avg TSB   Change
  --------------------------------------------------
  2026-05-18          0         0     45.6     base
  2026-05-11        166         2     45.5     base
  2026-05-04        279         5     28.6   +67.7% ⚠️
  2026-04-27        492         5     13.7   +76.6% ⚠️
  2026-04-20        607         6      0.6   +23.4% ⚠️
  2026-04-13        686         5    -16.0   +12.9% ⚠️
```

Section 7 — Memory / RAG

Demonstrates: ChromaDB vector store with three collections, semantic retrieval, how workout embeddings capture interval intensity (not just overall pace).

ChromaDB stores 180 days of workout summaries as natural language embeddings using `gemini-embedding-001`. Structured sessions include interval split summaries so the embedding captures rep-level pace, not just the diluted overall average.

```
import requests

def embed(text):
    url  = f'https://generativelanguage.googleapis.com/v1beta/models/gemini-embedding-001:embedContent?key={GEMINI_API_KEY}'
    resp = requests.post(url, json={'model': 'models/gemini-embedding-001', 'content': {'parts': [{'text': text}]}})
    return resp.json()['embedding']['values']

print("=== ChromaDB Collections ===")
```

```
for name in ['workout_summaries', 'athlete_profile', 'session_notes']:
    try:
        col = mem_client.get_collection(name)  # No embedding_function
        print(f"  ✅ {name:<25} {col.count():>4} documents")
    except Exception as e:
        print(f"  ❌ {name}: {e}")

print("\n=== RAG Demo: Retrieving similar hard sessions ===")
print("Query: 'hard interval session fast rep pace'\n")

col    = mem_client.get_collection('workout_summaries')
qvec   = embed('hard interval session fast rep pace')
results = col.query(query_embeddings=[qvec], n_results=3)
for doc, meta, dist in zip(results['documents'][0], results['metadatas'][0], results['distances'][0]):
    print(f"[{meta['date']}] similarity: {1-dist:.3f}")
    print(doc)
    print()
```

```
=== ChromaDB Collections ===
  ✅ workout_summaries          115 documents
  ✅ athlete_profile              9 documents
  ✅ session_notes                1 documents

=== RAG Demo: Retrieving similar hard sessions ===
Query: 'hard interval session fast rep pace'

[2026-04-08] similarity: 0.737
Date: 2026-04-08 | Type: vo2max | Distance: 14.48km | Duration: 66.4min
Overall avg pace: 4:35/km | Vo2max target: 3:40/km. Actual: 4:35/km (55s slower than target). Slightly under — check conditions,
HR: 146.8 avg / 174 max | Elevation: 107.0m
Training load: 319.21466064453125 | Aerobic effect: 3.9000000953674316 | Anaerobic effect: 3.5 | Suffer score: 60
Interval splits: Hard efforts: 9 reps, avg 3:31/km, fastest 3:26/km | Recovery/easy laps: 8, avg 6:38/km
Recovery context: HRV status: UNBALANCED, sleep: 7.3h, readiness: 72.0, resting HR: 40.0 bpm
Garmin enriched: yes

[2026-03-11] similarity: 0.715
Date: 2026-03-11 | Type: unclassified | Distance: 14.28km | Duration: 62.48min
Overall avg pace: 4:23/km | Cannot evaluate — target pace or actual pace is missing.
HR: 156.4 avg / 177 max | Elevation: 217.0m
Training load: 241.42921447753906 | Aerobic effect: 4.599999904632568 | Anaerobic effect: 0.4000000059604645 | Suffer score: 98
Recovery context: HRV status: BALANCED, sleep: 7.6h, readiness: 87.0, resting HR: 37.0 bpm
Garmin enriched: yes

[2025-12-20] similarity: 0.713
Date: 2025-12-20 | Type: unclassified | Distance: 18.02km | Duration: 78.9min
Overall avg pace: 4:23/km
HR: 156.7 avg / 176 max | Elevation: 178.0m
Training load: 262.9627685546875 | Aerobic effect: 4.599999904632568 | Anaerobic effect: 2.0 | Suffer score: 127
Recovery context: HRV status: BALANCED, sleep: 7.0h, readiness: 78.0, resting HR: 45.0 bpm
Garmin enriched: yes
```

```
def embed(text):
    url  = f'https://generativelanguage.googleapis.com/v1beta/models/gemini-embedding-001:embedContent?key={GEMINI_API_KEY}'
    resp = requests.post(url, json={'model': 'models/gemini-embedding-001', 'content': {'parts': [{'text': text}]}})
    return resp.json()['embedding']['values']

print("=== RAG Demo: Retrieving similar hard sessions ===")
print("Query: 'hard interval session fast rep pace'\n")

col     = mem_client.get_collection('workout_summaries')
qvec    = embed('hard interval session fast rep pace')
results = col.query(query_embeddings=[qvec], n_results=3)
for doc, meta, dist in zip(results['documents'][0], results['metadatas'][0], results['distances'][0]):
    print(f"[{meta['date']}] similarity: {1-dist:.3f}")
    print(doc)
    print()
```

```
=== RAG Demo: Retrieving similar hard sessions ===
Query: 'hard interval session fast rep pace'

[2026-04-08] similarity: 0.737
Date: 2026-04-08 | Type: vo2max | Distance: 14.48km | Duration: 66.4min
Overall avg pace: 4:35/km | Vo2max target: 3:40/km. Actual: 4:35/km (55s slower than target). Slightly under — check conditions,
HR: 146.8 avg / 174 max | Elevation: 107.0m
Training load: 319.21466064453125 | Aerobic effect: 3.9000000953674316 | Anaerobic effect: 3.5 | Suffer score: 60
Interval splits: Hard efforts: 9 reps, avg 3:31/km, fastest 3:26/km | Recovery/easy laps: 8, avg 6:38/km
Recovery context: HRV status: UNBALANCED, sleep: 7.3h, readiness: 72.0, resting HR: 40.0 bpm
Garmin enriched: yes
```

```
[2026-03-11] similarity: 0.715
Date: 2026-03-11 | Type: unclassified | Distance: 14.28km | Duration: 62.48min
Overall avg pace: 4:23/km | Cannot evaluate — target pace or actual pace is missing.
HR: 156.4 avg / 177 max | Elevation: 217.0m
Training load: 241.42921447753906 | Aerobic effect: 4.599999904632568 | Anaerobic effect: 0.4000000059604645 | Suffer score: 98
Recovery context: HRV status: BALANCED, sleep: 7.6h, readiness: 87.0, resting HR: 37.0 bpm
Garmin enriched: yes

[2025-12-20] similarity: 0.713
Date: 2025-12-20 | Type: unclassified | Distance: 18.02km | Duration: 78.9min
Overall avg pace: 4:23/km
HR: 156.7 avg / 176 max | Elevation: 178.0m
Training load: 262.9627685546875 | Aerobic effect: 4.599999904632568 | Anaerobic effect: 2.0 | Suffer score: 127
Recovery context: HRV status: BALANCED, sleep: 7.0h, readiness: 78.0, resting HR: 45.0 bpm
Garmin enriched: yes
```

```
print("=== Full Memory Context (as injected into agent prompt) ===")
print("Query: 'how did my recent threshold sessions go?'\n")

query = 'how did my recent threshold sessions go?'
qvec  = embed(query)
lines = ['=== Memory Context ===']

# Profile
try:
    pc  = mem_client.get_collection('athlete_profile')
    pr  = pc.query(query_embeddings=[qvec], n_results=2)
    if pr['documents'][0]:
        lines += ['\n--- Athlete Profile ---', '\n\n'.join(pr['documents'][0])]
except Exception as e:
    print(f"Profile skipped: {e}")

# Workouts
try:
    wc  = mem_client.get_collection('workout_summaries')
    wr  = wc.query(query_embeddings=[qvec], n_results=2)
    if wr['documents'][0]:
        lines.append('\n--- Relevant Past Workouts ---')
        for doc in wr['documents'][0]:
            lines += [doc, '']
except Exception as e:
    print(f"Workouts skipped: {e}")

# Notes
try:
    nc  = mem_client.get_collection('session_notes')
    if nc.count() > 0:
        nr  = nc.query(query_embeddings=[qvec], n_results=1)
        if nr['documents'][0]:
            lines.append('--- Session Notes ---')
            for doc, meta in zip(nr['documents'][0], nr['metadatas'][0]):
                lines.append(f"[{meta.get('date','')}] {doc}")
except Exception as e:
    print(f"Notes skipped: {e}")

print('\n'.join(lines))
```

```
=== Full Memory Context (as injected into agent prompt) ===
Query: 'how did my recent threshold sessions go?'

=== Memory Context ===

--- Athlete Profile ---
Agent notes: When evaluating workouts, always compare against coach-prescribed targets, not generic benchmarks. Pace tolerance i

Training paces (coach-prescribed):
  easy: By feel — anything slower than 4:45/km
  marathon: 4:14/km
  threshold: 4:01/km
  1hr: 3:56/km
  fartlek: 3:49/km
  8k: 3:46/km
  vo2max: 3:40/km
  hm_race: ~3:59/km (for 1:24 target)

--- Relevant Past Workouts ---
Date: 2026-04-08 | Type: vo2max | Distance: 14.48km | Duration: 66.4min
Overall avg pace: 4:35/km | Vo2max target: 3:40/km. Actual: 4:35/km (55s slower than target). Slightly under — check conditions,
```

```
HR: 146.8 avg / 174 max | Elevation: 107.0m
Training load: 319.21466064453125 | Aerobic effect: 3.9000000953674316 | Anaerobic effect: 3.5 | Suffer score: 60
Interval splits: Hard efforts: 9 reps, avg 3:31/km, fastest 3:26/km | Recovery/easy laps: 8, avg 6:38/km
Recovery context: HRV status: UNBALANCED, sleep: 7.3h, readiness: 72.0, resting HR: 40.0 bpm
Garmin enriched: yes

Date: 2025-11-07 | Type: easy | Distance: 4.24km | Duration: 20.55min
Overall avg pace: 4:51/km | Easy run pace of 4:51/km is within easy territory (above 4:45/km ceiling). ✅
HR: 136.9 avg / 163 max | Elevation: 21.0m
Training load: 63.93414306640625 | Aerobic effect: 2.700000047683716 | Anaerobic effect: 0.10000000149011612 | Suffer score: 9
Recovery context: HRV status: LOW, sleep: 9.7h, readiness: 53.0, resting HR: 47.0 bpm
Garmin enriched: yes

--- Session Notes ---
[2026-05-02] # Your note here — no length limit
Last Saturday long run before Freddy Half. 18mins at 1HR pace. 2mins recovery jog and then 5mins at Fartlek
```